EECS 485 Midterm Exam Fall 2025 SOLUTION

This is a 120 minute, closed-book exam.

You may use one cheat sheet, 8.5”x11”, double-sided.

No other notes or use of any computing or communication hardware (e.g., calculators, cell phones, smart glasses, etc.) is allowed.

For multiple-choice questions, indicate your response by filling in the appropriate bubble. Fill in the bubble completely, and make sure to erase completely if you change your answer. For free response questions, write your answers clearly in the space provided in each problem.

Collaboration or assistance is prohibited

- You are NOT allowed to collaborate with anyone else to answer these questions.
- You are NOT allowed to solicit assistance from anyone to answer these questions.

You are to abide by the University of Michigan/Engineering Honor Code. To receive a grade, please sign below to signify that you have kept the honor code pledge.

I have neither given nor received aid on this exam, nor have I concealed any violations of the Honor Code.

Signature: _____

Name: _____

Uniqname: _____

UMID: _____

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Multiple Choice (40 points)

Demo Question: How should I fill in my responses for a Multiple Choice question?

- **A. Fill in the chosen bubble completely and only select ONE per question!**

HTML, CSS, HTTP, Server-Side & Client-Side Dynamic Page

---

1. Which of the following works best as a server-side dynamic page?

   ◯ A. A site with a stopwatch that keeps time in milliseconds

   ◯ B. A live-updating food delivery tracking service

   ◯ C. A site that hosts playable archives of old video games

   ◯ D. A forum for discussing films

   ◯ E. A site that visualizes the age of the universe

Solution: Option A would be a client-side dynamic page since the server always responds with the same dynamic page data. C and E are static pages. B would need to update without refreshing, which wouldn't work for server-side dynamic pages. D would work well, since it rarely needs to refresh and updates don't need to be seen immediately.

2. All of the following result in a 4XX HTTP error code **EXCEPT:**

   ◯ A. Getting redirected to a different route

   ◯ B. Attempting to access a route that no longer exists

   ◯ C. Performing an action with unauthorized access

   ◯ D. Making a POST request to modify another user's data

   ◯ E. Not being logged into your account.

Solution: Redirection leads to a 3XX error code, making A false. B is a case of accessing a resource that no longer exists, a 404 error, and C, D, and E all have to do with unauthorized access, a 403 error.

3. Which is **FALSE** about HTTP?

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

- ◯ A. Cookies can be freely modified by users within the browser
- ◯ B. It's a request-response protocol for sending data between hosts
- ◯ C. HTTP file servers are on the server-side for static pages
- ◯ D. After connection, server sends data without new requests.
- ◯ E. The HEAD request can help verify if a page has been changed

Solution: D is false as the server will not send an HTTP response without an associated request from the client.

4. You're a developer for the new movie reviewing platform film485. Which flask route will run after submitting this form?

```
<form action="/film485/review/?target=/dramas/" method="post">
     <input type="text" name="review"/>
     <input type="number" name="score" min="1" max="10"/>
     <input type="hidden" name="movie_id"
          value="Casablanca"/>
     <input type="submit" name="submit_review" value="submit"/>
</form>
```

- ◯ A. @flask.app.route('/film485/review/?target=/dramas/', methods=["GET"])
- ◯ B. @flask.app.route('/film485/review/', methods=["POST"])
- ◯ C. @flask.app.route('/film485/review/?target=/dramas/', methods=["POST"])
- ◯ D. @flask.app.route('/film485/Casablanca/submit_order', methods=["POST"])
- ◯ E. None of the above

Solution: B is the only correct option, as it is performing a POST request as defined by the form "method" parameter and has the endpoint of "/film485/review", the route pointed to by the action parameter.

5. Which of the following statements about JavaScript is **FALSE**?

- ◯ A. Out of HTML, CSS, and JavaScript, it is the only programming language
- ◯ B. It can be used to change the Document Object Model without reloading the page
- ◯ C. Provides the same functionality as Python for web development

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ D. It is the only programming language understood by web browsers

◯ E. JavaScript is an event-driven programming language

Solution: A is true since HTML is a markup language and CSS is a stylesheet language, B is also true since JS updates the page in response to real-time events. C is false as Python cannot be understood by the browser, and thus can't modify the DOM. D is true.

Sessions, Encryption, Web Security

---

6. What is the primary purpose of sessions in web applications?

◯ A. To permanently store user passwords on a server

◯ B. To generate cookies that contain sensitive data

◯ C. To preserve state between HTTP requests

◯ D. To compress data sent over the network

◯ E. To encrypt data sent over the network

Solution: C is correct. HTTP is stateless, so sessions act as an extra "id" between requests from the same user. In general, you should never store sensitive data in cookies, because they are stored on client-side and can be sued maliciously if in the hands of an attacker. Sessions are not related to how data is sent over the network.

7. Which of the following statements about encryption is **FALSE**?

◯ A. AES encryption cannot be reversed, even with the correct key

◯ B. The same key is used for both encryption and decryption in symmetric encryption

◯ C. Each party has both a public and private key in asymmetric encryption

◯ D. Asymmetric encryption provides sender authenticity

◯ E. Public keys are distributed safely via certificates

Solution: A is correct. AES is a symmetric encryption algorithm, and symmetric encryption is a reversible process due to encryption and decryption using the same key.

8. Which of the following statements is **NOT** an attack to cracking passwords?

◯ A. Brute force

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ B. Dictionary

◯ C. Rainbow tables

◯ D. Salting

◯ E. John the Ripper

Solution: D is correct. “Salting” is a way of altering the way a password is hashed by appending a random number to the plaintext password before hashing it. Performing/using brute force attacks, dictionary attacks, rainbow tables, and John the Ripper are all ways to crack passwords.

9. What is the main purpose of CSRF tokens?

◯ A. Encrypt session cookies

◯ B. Hash user passwords

◯ C. Prevent SQL injections

◯ D. Prevent tricking a user into unknowingly triggering malicious requests

◯ E. Log user activity

Solution: D is correct. CSRF tokens protect authenticated users from attackers that may trick the user into unknowingly submitting a malicious request (also known as a cross-site request forgery attack) by validating the sender of the request. CSRF tokens are not related to encrypting session cookies, hashing passwords, SQL injection protection, or logging user activity.

10. Which of the following best describes a defense to Cross-site scripting (XSS) attacks?

◯ A. Storing plaintext passwords in cookie for verification

◯ B. Escape all user input

◯ C. Deleting all cookies

◯ D. Being on private browser mode

◯ E. Blocking all cookies sent to the browser

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Solution: B is correct. Escaping user input protects the browser from unintentionally executing malicious hidden code. For example, Flask's render_template() escapes HTML, which stops injected scripts from executing. Sensitive information should never be stored in cookies, and cookies as well as private browsing mode do not prevent XSS attacks.

REST APIs, JavaScript, Asynchronous Programming

---

11. Alex is trying to update his favorite post on Insta485 while sipping his matcha. He sends a PATCH request to /api/v1/posts/7/ to update the image URL, but also induces an unsupported field "likes":1000. The server responds with 200, so what happened?

    ◯ A. The entire post was updated

    ◯ B. The image URL was updated, including the "likes" field

    ◯ C. The server created a new post with the JSON body

    ◯ D. The request failed silently, leaving the post unchanged

    ◯ E. The image URL was updated, and the "likes" field was ignored

Solution: 200 indicates the operation succeeded, so the post was updated. A is wrong → PATCH does not replace the entire object (that would be PUT). B is wrong → PATCH only updates the fields which the API allows, and extra fields are ignored. C is wrong → PATCH does not create new posts; that's POST. D is wrong → the request succeeded (200 OK).

12. Which of the following statements about GET or POST requests is TRUE?

    ◯ A. GET requests are intended to create or modify resources on the server

    ◯ B. POST requests are idempotent by default

    ◯ C. GET requests are safe, meaning they should not change server state

    ◯ D. POST requests are always cacheable by default

    ◯ E. GET requests require a JSON body to function

Solution: A is wrong because GET requests are supposed to retrieve data without modifying anything on the server. B is wrong because POST requests are used to create resources and

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

are not idempotent (multiple identical POSTs may create multiple resources). D is wrong because POST requests are not always cacheable by default. E is wrong because GET requests do not require a JSON body.

13. Which is TRUE about console and document APIs?

- ◯ A. console writes to the web page
- ◯ B. document API allows JS to change the DOM dynamically
- ◯ C. console can modify HTML elements
- ◯ D. document writes to the developer console
- ◯ E. All of the above are false

Solution: document represents the DOM of the page. Using it, JS can read or modify elements, add/remove nodes, change text, attributes, etc., which updates the rendered page dynamically. console only writes to the developer console (not the web page), so A and C are wrong. document writes to the web page (modifies the DOM), not the developer console, so D is wrong. E is wrong because B is true.

14. Why is asynchronous programming important in JavaScript?

- ◯ A. It allows JavaScript to run on multiple threads by default.
- ◯ B. It prevents blocking of the main thread during slow operations (e.g., network requests).
- ◯ C. It guarantees that all operations run strictly in sequence.
- ◯ D. It makes individual API calls faster.
- ◯ E. It forces the browser to reload the DOM after every change.

Asynchronous programming allows JS to continue executing other code while waiting for slow operations to complete. A is wrong because JS doesn't automatically run on multiple threads. C is wrong because async does not guarantee strict ordered execution. D is wrong because async doesn't make network requests faster; it just avoids blocking. E is wrong because async has nothing to do with reloading the DOM.

15. David runs this code:

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
console.log("Number");
fetch('/api/random/int')
      .then(resp => console.log(Math.random()));
console.log("Done");
```

What is the output order?

○ A. Number → {random number} → Done

○ B. {random number} → Number → Done

○ C. Done → Number → random number

○ D. Number → Done → {random number}

◯ E. None of the above

1. console.log("Number") runs first → prints Number.

2. fetch('/api/random/int') is asynchronous and returns a promise. The .then() callback is scheduled to run after current call stack is empty.

3. console.log("Done") runs immediately after the fetch is called → prints Done.

4. Once the network request completes, the .then() callback executes → prints a random number.

Distributed Systems, MapReduce, Google File System

16. Sydney is trying to run a MapReduce program when a reducer crashes midway through processing its input. How does MapReduce ensure correctness?

◯ A. All mappers re-run to regenerate the lost input for the reducer

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ B. A singular mapper gets re-run to regenerate the lost input

◯ C. Another reducer that didn't crash picks up the task

◯ D. The reducer restarts using stored mapper outputs

◯ E. The job is aborted and must restart from scratch

Solution: D, mapping outputs get stored locally, so you don't need to rerun any mappers. Reducers can directly restart their job with the stored values. Therefore, A, B, C, and E are all incorrect.

17. Suppose a chunkserver with replicas suddenly crashes and does not send heartbeat messages to the primary. Which of the following **BEST** describes how GFS maintains fault tolerance in this situation?

◯ A. The primary immediately shuts down the entire system until the chunkserver recovers

◯ B. The primary automatically reassigns clients to fetch data from the primary's local copy of the chunk

◯ C. The primary detects the failure and instructs other chunkservers to create new replicas of the affected chunks

◯ D. The primary promotes another chunkserver to become the new primary to avoid losing data

◯ E. The GFS system gets restarted

Solution: C, GFS maintains fault tolerance by monitoring chunkserver heartbeats; when a chunkserver stops sending them, the master assumes it has failed and re-replicates its chunks on healthy servers. This ensures the replication factor stays consistent even after failures.

18. Soham is trying to analyze different distributed systems. Which option best captures a key strength and weakness of GFS?

◯ A. Strength: Optimized for small files
Weakness: Cannot replicate data across chunkservers

◯ B. Strength: High throughput for large sequential reads and writes
Weakness: High latency and inefficiency for small files

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

◯ C. Strength: Provides strict sequential consistency

Weakness: Cannot tolerate failures of the primary

◯ D. Strength: Primary node never becomes a bottleneck

Weakness: Metadata cannot be recovered after a primary failure

◯ E. Strength: Low-latency interactive performance

Weakness: Overhead from using expensive, highly reliable servers

Solution: B, GFS is optimized for large, batch workloads (e.g., web crawling), but suffers on small-file workloads. A is wrong because it isn't optimized for small files. C is wrong because it can tolerate failures of the primary. D is wrong because the primary is a potential bottleneck. E is wrong because GFS is built for throughput, not low latency.

19. The EECS 485 Staff wants to write a MapReduce program to calculate the weighted average rating for a set of movies, where each input record has the format (movie_id, rating, num_votes). Each rating itself has a number of votes, so the weighted average for a movie can be calculated by

$$weighted\ rating = \frac{\sum(rating * num\ votes)}{\sum num\ votes}$$

Which of the following must the mapper output so that the reducer can correctly compute the weighted rating?

◯ A. (movie_id, num_votes)

◯ B. (movie_id, rating*num_votes)

◯ C. (movie_id, rating)

◯ D. (movie_id, rating*num_votes, num_votes)

◯ E. (num_votes, rating)

Solution: To compute the weighted average rating per movie, the reduce needs both the sum of (rating x num_votes) and the sum of num_votes grouped by movie_id. Therefore the mapper must emit all three (option D). Options A-C lack necessary data for this calculation. During the exam, some people were told that it was the weighted rating for the entire set, in which case E would coincidentally be correct. However note that this is not correct for the problem, since the num_votes and ratings aren't grouped by a movie_id key.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

20. Alex, Risha, and Garv are all writing to a blank GFS file. At the same time, Risha writes "six" and Alex writes "seven" to the same chunk. Then, Garv decides to open the document. If that file region is guaranteed to be defined, what might Garv see?

- ◯ A. six
- ◯ B. seven
- ◯ C. six seven
- ◯ D. A or B
- ◯ E. A, B, or C

Solution: D. It can't be C because the system is defined, so GFS guarantees that the final state will reflect one complete write or the other, but never an inconsistent mixture of both. Therefore it must either be A or B.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 1: ~~NFL~~ 485 Draft (30 points)

It's football season! The EECS 485 staff want to create a website for a simplified fantasy football game where each user (a manager) drafts a five-player team (1 quarterback (QB), 1 running back (RB), 1 wide receiver (WR), 1 tight end (TE), and 1 kicker (K)). Managers may draft players one at a time or select multiple players at once.

We have provided the following database and a helper function:

```
CREATE TABLE managers (
  manager_id INTEGER AUTOINCREMENT,
  username VARCHAR(20) NOT NULL,
  team_name VARCHAR(20) NOT NULL,
  is_turn BOOLEAN NOT NULL,
  PRIMARY KEY (manager_id)
);
```

```
CREATE TABLE players (
  player_id INTEGER AUTOINCREMENT,
  name VARCHAR(20) NOT NULL,
  filename VARCHAR(64) NOT NULL,
  position VARCHAR(2) NOT NULL,
  drafted_manager_id INTEGER,
  PRIMARY KEY (player_id)
  FOREIGN KEY(drafted_manager_id)
      REFERENCES managers(manager_id)
);
```

`get_upload(filename):`

```
# serve one file in the uploads directory
```

Simplifying assumptions:

- You can assume that the only positions in the players table are 'QB', 'RB', 'WR', 'TE', 'K'

find_free_agents

Returns a list of dictionaries for undrafted players (player_id, name, filename, position) in positions not already filled by the manager, ordered by descending player_id.

```
def find_free_agents(drafted_manager_id):
      con = get_db_connection()
      free_agents = con.execute("""
            SELECT player_id, name, filename, position
            FROM players
            WHERE drafted_manager_id IS NULL
              AND position NOT IN (
                SELECT DISTINCT position
                FROM players
                WHERE drafted_manager_id = ?
              )
            ORDER BY player_id DESC
            """, (drafted_manager_id,)).fetchall()
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
    if not free_agents:
        return []
    return free_agents
```

GET '/draft/<manager_id>/'

The '/draft/<manager_id>/' route renders the "waiting.html" template with no context if the boolean is_turn for the logged-in user (aka the Manager) is false, otherwise "index.html" with a context dictionary. The context must include:

- logname: The username of the currently logged-in user.
- manager_id: The ID of the logged-in user's team.
- team_name: The name of the logged-in user's team.
- drafted_players: A list of dictionaries of the logged-in user's current team with dictionary parameters player_id, name, filename, position
- has_positions_left: A boolean that is ~~true~~ false when length of drafted players is 5, ~~false~~ true otherwise. Since the original verbiage was incorrect, we accepted both true or false. For students to get full points all that was required was the expression evaluated to a bool.
- free_agents: A list of dictionaries of the undrafted players [player_id, name, filename, position]

NOTE: The username of the logged-in user is stored in a session with the key "logname"

```
@app.route('/draft/<manager_id>/')
def show_index(manager_id):
    con = get_db_connection()
```

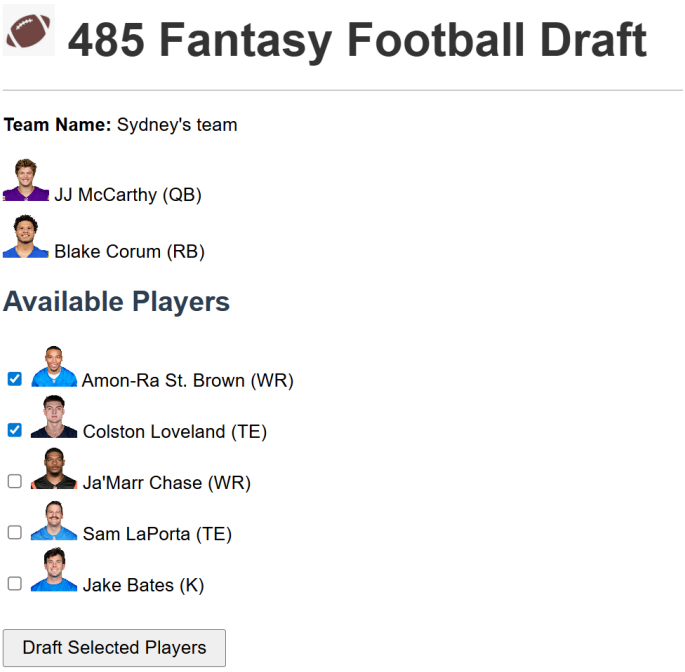
```
    logname = flask.session["logname"]
    manager = con.execute("""
        SELECT team_name, is_turn
        FROM managers
        WHERE username = ?
        """, (logname,)).fetchone()
    if not manager:
        flask.abort(403)
    if not manager["is_turn"]:
        return flask.render_template("waiting.html")
    drafted_players = con.execute(
        "SELECT name, position, filename FROM players WHERE
        drafted_manager_id = ?",
        (manager_id,)
    ).fetchall()
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
    free_agents = find_free_agents(manager_id)

    context = {
        "logname": logname,
        "manager_id": manager_id,
        "team_name": manager["team_name"],
        "drafted_players": drafted_players,
        "has_positions_left": len(drafted_players) < 5,
        "free_agents": free_agents
    }
    return flask.render_template(
        "index.html",
        **context,
    )
```

index.html



Write the <body> section of a Jinja template for the draft page. It should include:

- The logged-in user's team name
- The logged-in user's current drafted team, showing each player's profile photo, name, and position.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

- If the logged-in user still has positions left to fill, display a list of all available (undrafted) players with their name and position.
- A form that allows the user to draft **one or more** players onto their team.
  - The form should make a POST request to /draft_player/ ~~with the action set to the current URL~~. We accepted any solution that sent a POST request to /draft_player/, even if there were extra parameters or a target.
  - The form arguments must include both the player_id and the manager_id.

Hints:

- Use <input type="checkbox"> to let the user select multiple options. When all checkboxes share the same "name" (e.g., player_id), Flask lets you retrieve the selected values as a list with `request.form.getlist("player_id")`.

```
<body>
<p>Team Name: {{ team_name }}</p>

{% for player in drafted_players %}
    <p> <img src="{{url_for('get_upload', filename= player.filename)}}">
     {{ player.name }} {{ player.position }}
    </p>
{% endfor %}


{% if has_positions_left %}
  <h2>Available Players</h2>
  <form action="/draft_player/" method="post">
    <input type="hidden" name="manager_id" value="{{ manager_id }}">

    {% for player in free_agents %}
      <input type="checkbox" name="player_id" value="{{ player.player_id }}">
      <img src="{{url_for('get_upload', filename= player.filename)}}">
      {{ player.name }} ({{ player.position }})
      <br>
    {% endfor %}

    <br>
    <input type="submit" value="Draft Selected Players">
  </form>
{% endif %}

</body>
```

POST '/draft_player/'

This route accepts a POST request which gets each player from the user form input, and adds them to the logged-in user's team.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

- Redirect to show_index
- When drafting,
  - if the user submits more players than positions available for a given role (for example, drafts 2 QBs when only 1 QB slot is available), only the first valid player for each position should be drafted. Any additional players for that position should be ignored.
  - if the user submits a player for a position they already have keep the drafted player and ignore submission.
- You can assume the manager_id will be valid and corresponds to the logged-in user's team.

Hint:

- If your Flask form has multiple inputs with the same name (e.g., checkboxes named player_id), you can retrieve all of their values as a list of **strings** with order maintained using:

  `player_ids = flask.request.form.getlist("player_id")`
- Using 'find_free_agents' may be useful in determining if a position/player had already been drafted

```
@app.route('/draft_player/', methods=["POST"])
def handle_post():
    con = get_db_connection()
```

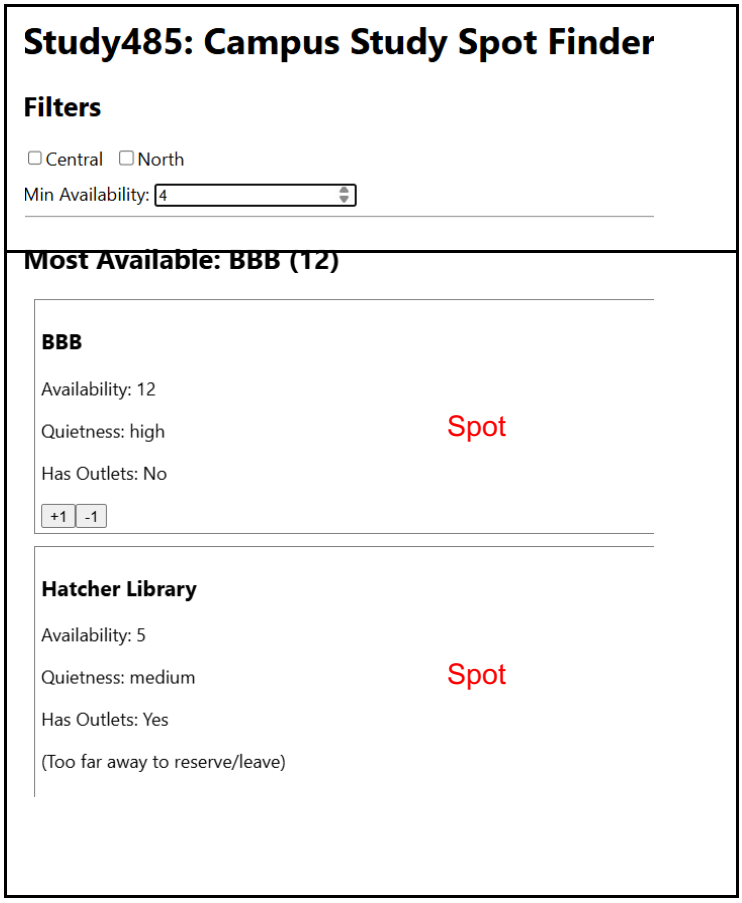
```
    m_id = int(flask.request.form["manager_id"])
    player_ids = [int(p) for p in flask.request.form.getlist("player_id")]
    free_agents = find_free_agents(m_id)
    free_agents_by_id = {p['player_id']: p for p in free_agents}
    drafted_positions = set()

    for pid in player_ids:
        player = free_agents_by_id.get(pid)
        pos = player['position']
        if (not player) or (pos in drafted_positions):
            continue

        con.execute("""
            UPDATE players
            SET drafted_manager_id = ?
            WHERE player_id = ?
            """,
            (m_id, pid)
        )
        drafted_positions.add(pos)
    return flask.redirect(flask.url_for('show_index', manager_id=m_id))
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Free Response 2: Study485 (30 points)

Implement a campus study spot finder, using React functional components that allows students to see availability of study spots and report when seats open or fill up. Implement the **Campus** and **Spot** functional components.



Implemented for you

Campus Component

`Campus` component:

- Declare **3** state variables:
  - `spots` (list) stores the list of study spots from the API.
  - `locationFilter` (string) representing `"central"`, `"north"`, or `""` for both.
  - `availabilityFilter` (integer) representing the minimum availability to show.
- On initial render, make a **GET** request to `/api/v1/spots/?location=<string>&minAvailability=<int>` and store results in `spots`.
- When handling the API response:
  - **Filter out** any spots with availability < 0, indicating the spot is closed.
- Renders a list of `Spot` components, one for each study spot, with the right props (specified in the `Spot` component section).

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

- Computes the spot with the **highest availability** and renders its name and availability.
- Declares a provided function `handleReport(id, change)` that makes a **POST** request to `/api/v1/spots/<id>/report/` and updates the availability of that spot (**implemented for you**).

API routes:

Note: You should correctly handle any errors thrown by API calls to the backend service. You should print a reasonable error message to the console.

GET /api/v1/spots/?location=<string>&minAvailability=<int>

This API endpoint accepts GET requests and returns a list of study spots.

Returns

```
[
  { "id": 1, "name": "Hatcher Library", "availability": 5,
"location": "central" },
  { "id": 2, "name": "BBB", "availability": 12, "location":
"north" },
...
]
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

Campus Component

```
import React, { StrictMode, useEffect, useState } from "react";
import Spot from “./Spot”;
export default function Campus(){
   // Declare state variables
```

```
const [spots, setSpots] = useState([]);
const [locationFilter, setLocationFilter] = useState("");
const [availabilityFilter, setAvailabilityFilter] = useState(0);
```

```
   const handleReport = (id, change) => {
      // Makes POST request to update availability
   };

   // Fetch spots on initial render
   useEffect(() => {
```

```
let ignoreStaleRequest = false;
fetch(      `/api/v1/spots/?location=${locationFilter}&minAvailab
ility=${availabilityFilter}`
        )
        .then((response) => {
             if (!response.ok) throw Error(response.statusText);
                return response.json();
         })
        .then((data) => {
                if (!ignoreStaleRequest) {
                const openSpots = data.filter((s) =>
s.availability >= 0);
                setSpots(openSpots);
                }
         })
        .catch((error) => console.log(error));
return () => {
        ignoreStaleRequest = true;
};
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
}, [locationFilter, availabilityFilter]);
```

```
let mostAvailable = null;
for (const s of spots) {
        if (mostAvailable === null || s.availability >
mostAvailable.availability) {
            mostAvailable = s;
        }
}
```

```
// compute mostAvailable
```

```
return (
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
<>
// Title and Filters implemented for you
<h1>Study485: Campus Study Spot Finder</h1>
<h2>Filters</h2>
<div> ... </div>

            {mostAvailable && (
                <h2>
                    Most Available: {mostAvailable.name}
({mostAvailable.availability})
                </h2>
            )}

            {spots.map((s) => (
                <Spot
                    key={s.id}
                    id={s.id}
                    name={s.name}
                    availability={s.availability}
                    handleReport={handleReport}
                />
            ))}
</>
```

```
    );
};

export default Campus;
```

`Spot` component:

- Takes 4 props:
  - `id` (integer), `name` (string), `availability` (integer), and `handleReport` (function).
- Declares **2** state variables (**implemented for you**):
  - `quietness` (string)
  - `hasOutlets` (boolean)
- On initial render, makes a **GET** request to `/api/v1/spots/<id>/details/` and stores the returned values in state (**implemented for you**).
- Renders the spot's name, availability, quietness, and if it has outlets ("Yes"/"No")
- Uses a utility helper function `distanceToUser(id)` that returns distance to the spot.

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

- If distance ≤ 100m: render `+1` and `-1` buttons that call `handleReport(id, change)`.
- Else: render a message `(Too far away to reserve/leave)`.

Spot Component

```
import React, { StrictMode, useEffect, useState } from "react";

export default function Spot({
id, name, availability, handleReport
}){
   // State variables declared and initialized for you
   const [quietness, setQuietness] = ...;
   const [hasOutlets, setHasOutlets] = ...;

   // Fetch details on initial render
   useEffect(() => {
        // Assume this fetch works and correctly sets state.
   }, []);
```

```
return (
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/

uniqname: _____

```
<div>
            <h3>{name}</h3>
            <p>Availability: {availability}</p>
            <p>Quietness: {quietness}</p>
            <p>Has Outlets: {hasOutlets ? "Yes" : "No"}</p>

            {distanceToUser(id) <= 100 ? (
                <>
<button onClick={() => handleReport(id, +1)}>+1</button>
<button onClick={() => handleReport(id, -1)}>-1</button>
                </>
            ) : (
                <p>(Too far away to reserve/leave)</p>
            )}
</div>
```

```
    );
};

export default Spot;
```

Licensed under Creative Commons Attribution-NonCommercial 4.0 License.
https://creativecommons.org/licenses/by-nc/4.0/